

Module 19 — Practice Tasks

Task 1: `getFormattedPrices(prices)` — `map()`

Given an array of numbers, return a new array where each price is formatted as a string prefixed with "\$".

Input: `[100, 250, 75]`

Output: `["$100", "$250", "$75"]`

Task 2: `getAvailableProducts(products)` — `filter()`

Given an array of product objects `{ name, stock }`, return only the products where `stock` is greater than 0.

Input:

`[{ name: "Pen", stock: 5 }, { name: "Bag", stock: 0 }, { name: "Notebook", stock: 2 }]`

Output:

`[{ name: "Pen", stock: 5 }, { name: "Notebook", stock: 2 }]`

Task 3: `findProductByName(products, name)` — `find()`

Given an array of product objects, return the first product matching the given name, or `undefined` if not found.

Input:

```
products = [{ name: "Pen", price: 10 }, { name: "Bag", price: 500 }], name = "Bag"
```

Output:

```
{ name: "Bag", price: 500 }
```

Task 4: `getTotalStockValue(products)` — `reduce()`

Given an array of products { `price`, `stock` }, return the total value of all stock combined ($\text{price} \times \text{stock}$, summed).

Input:

```
[{ price: 50, stock: 4 }, { price: 20, stock: 10 }]
```

Output: 400

Task 5: `getDiscountedTotalForCategory(products, category)` —

Chaining map + filter + reduce

Given an array of products { `name`, `category`, `price` }, return the total price of all products in the given category after applying a 10% discount to each.

Input:

```
products = [{ name: "Pen", category: "stationery", price: 100 }, { name: "Bag",  
category: "accessory", price: 500 }, { name: "Notebook", category: "stationery",  
price: 60 }], category = "stationery"
```

Output: 144

Task 6: `createIdGenerator()` — Closures

Return a function that generates a new sequential ID starting from 1 each time it is called, using an internal counter with no global variable.

Input:

```
const nextId = createIdGenerator();  
  
nextId();  
  
nextId();  
  
nextId();
```

Output: 1, 2, 3

Task 7: `sortByPriceAscending(products)` — Callback Function

Given an array of product objects { `name`, `price` }, return a new array sorted from lowest to highest price using a comparator callback in `sort()`.

Input:

```
[{ name: "Bag", price: 500 }, { name: "Pen", price: 10 }, { name: "Notebook", price: 60 }]
```

Output:

```
[{ name: "Pen", price: 10 }, { name: "Notebook", price: 60 }, { name: "Bag", price: 500 }]
```

Task 8: `applyBonusPoints(users, bonus)` — Pass by Reference vs Value

Return a NEW array of users with **points** increased by **bonus**, without mutating the original array or its objects.

Input: `users = [{ name: "Rafi", points: 20 }], bonus = 5`

Output: `[{ name: "Rafi", points: 25 }]`

(original array's object must still have **points: 20**)

Task 9: `isStrictMatch(input, target)` — Truthy/Falsy and ===

Return **true** only if **input** strictly equals **target** in both value and type, **false** otherwise.

Input: `isStrictMatch("18", 18)`

Output: `false`

Input: `isStrictMatch(18, 18)`

Output: `true`

Task 10: `getTopSellingProduct(orders)` — Mixed (map + reduce, capstone-style)

Given an array of orders `{ product, unitsSold }`,

return the name of the product with the highest total units sold across all orders (a product may appear in multiple orders).

Input: `[{ product: "Pen", unitsSold: 30 }, { product: "Bag", unitsSold: 12 }, { product: "Pen", unitsSold: 25 }]`

Output: "Pen"